

Lab 09 – ListBox, ComboBox, DatePicker

Title: Enhanced Input and Filtering with Specialized Controls

Objectives

This lab introduced more specialized WPF controls that handle specific data types and interactions. I needed to use a ComboBox for selecting from a predefined list of departments, a DatePicker for selecting dates with a calendar interface, and a ListBox for multiple-selection filtering by job function. These controls significantly improve the user experience compared to simple text inputs.

Implementation

MainWindow.xaml (Filter Section)

```
<!-- Filter Controls Section -->

<Border Grid.Row="0" Background="{StaticResource LightBrush}" Padding="10"
Margin="0,0,0,10" CornerRadius="5">

    <Grid>

        <Grid.ColumnDefinitions>

            <ColumnDefinition Width="*" />

            <ColumnDefinition Width="*" />

        </Grid.ColumnDefinitions>

        <StackPanel Grid.Column="0">

            <TextBlock Text="Filtrare angajați" FontWeight="Bold" FontSize="14"
Margin="0,0,0,10" />

            <!-- Tip Salariu RadioButtons -->
```

```

<GroupBox Header="Tip Salariu" Margin="0,0,0,10">

    <StackPanel Orientation="Horizontal">

        <RadioButton x:Name="rbToate" Content="Toate" IsChecked="True"
GroupName="TipFiltru" Margin="5" Checked="RadioButton_Checked"/>

        <RadioButton x:Name="rbLunar" Content="Lunar" GroupName="TipFiltru"
Margin="5" Checked="RadioButton_Checked"/>

        <RadioButton x:Name="rbOrar" Content="Orar" GroupName="TipFiltru" Margin="5"
Checked="RadioButton_Checked"/>

    </StackPanel>

</GroupBox>


<!-- Bonus CheckBox -->

<CheckBox x:Name="chkBonus" Content="Doar cu bonusuri" Margin="5"
Checked="CheckBox_Changed" Unchecked="CheckBox_Changed"/>


<!-- ListBox for Function Filter -->

<GroupBox Header="Funcție" Margin="0,10,0,0">

    <ListBox x:Name="lstFunctii" SelectionMode="Multiple" Height="100"
        SelectionChanged="LstFunctii_SelectionChanged">

        <ListBoxItem>Programator</ListBoxItem>

        <ListBoxItem>Manager</ListBoxItem>

        <ListBoxItem>Analist</ListBoxItem>

        <ListBoxItem>Designer</ListBoxItem>

        <ListBoxItem>Contabil</ListBoxItem>

        <ListBoxItem>Resurse Umane</ListBoxItem>

    </ListBox>

</GroupBox>


<Button Content="Aplică filtre" x:Name="btnFiltreaza" Click="BtnFiltreaza_Click"
    Background="{StaticResource AccentBrush}" Foreground="White"
    Padding="15,5" Margin="0,10" Width="120" HorizontalAlignment="Left"/>

</StackPanel>

```

```

<StackPanel Grid.Column="1" Margin="20,0,0,0">
    <TextBlock Text="Căutare rapidă" FontWeight="Bold" FontSize="14" Margin="0,0,0,10"/>

    <Label Content="Caută după nume:" Margin="0"/>

    <TextBox x:Name="txtCautareNume" TextChanged="TxtCautare_TextChanged"
Margin="0,0,0,10"/>


    <Label Content="Departament:" Margin="0"/>

    <ComboBox x:Name="cmbDepartamentFiltru" FontSize="14" Padding="5"
Margin="0,0,0,10"
        SelectionChanged="CmbDepartament_SelectionChanged">
        <ComboBoxItem>Toate</ComboBoxItem>
        <ComboBoxItem>IT</ComboBoxItem>
        <ComboBoxItem>Marketing</ComboBoxItem>
        <ComboBoxItem>Finante</ComboBoxItem>
        <ComboBoxItem>HR</ComboBoxItem>
        <ComboBoxItem>Productie</ComboBoxItem>
        <ComboBoxItem>Logistica</ComboBoxItem>
    </ComboBox>


    <Label Content="Data angajării (de la):" Margin="0"/>

    <DatePicker x:Name="dpDataAngajarii"
SelectedDateChanged="DpDataAngajarii_SelectedDateChanged"
        FontSize="14" Margin="0,0,0,10"/>

</StackPanel>

</Grid>

</Border>

```

Input Form with Specialized Controls

```

<!-- Input Form Section -->

```

```
<GroupBox Grid.Row="2" Header="Adăugare Angajat" Margin="0,10,0,0">
```

```
<Grid Margin="5">
```

```
<Grid.ColumnDefinitions>
```

```
<ColumnDefinition Width="Auto"/>
```

```
<ColumnDefinition Width="*/>
```

```
<ColumnDefinition Width="Auto"/>
```

```
<ColumnDefinition Width="*/>
```

```
</Grid.ColumnDefinitions>
```

```
<Grid.RowDefinitions>
```

```
<RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="Auto"/>
```

```
</Grid.RowDefinitions>
```

```
<!-- Row 0: Nume and Prenume -->
```

```
<Label Grid.Row="0" Grid.Column="0" Content="Nume:"/>
```

```
<TextBox Grid.Row="0" Grid.Column="1" x:Name="txtNume"  
TextChanged="ValidateForm"/>
```

```
<Label Grid.Row="0" Grid.Column="2" Content="Prenume:"/>
```

```
<TextBox Grid.Row="0" Grid.Column="3" x:Name="txtPrenume"  
TextChanged="ValidateForm"/>
```

```
<!-- Row 1: Funcție and Departament -->
```

```
<Label Grid.Row="1" Grid.Column="0" Content="Funcție:"/>
```

```
<TextBox Grid.Row="1" Grid.Column="1" x:Name="txtFuncție"  
TextChanged="ValidateForm"/>
```

```
<Label Grid.Row="1" Grid.Column="2" Content="Departament:"/>
```

```
<ComboBox Grid.Row="1" Grid.Column="3" x:Name="cmbDepartament" FontSize="14"  
Padding="5">
```

```

        <ComboBoxItem>IT</ComboBoxItem>

        <ComboBoxItem>Marketing</ComboBoxItem>

        <ComboBoxItem>Finante</ComboBoxItem>

        <ComboBoxItem>HR</ComboBoxItem>

        <ComboBoxItem>Productie</ComboBoxItem>

        <ComboBoxItem>Logistica</ComboBoxItem>

    </ComboBox>

    <!-- Row 2: Tip Salariu RadioButtons -->

    <Label Grid.Row="2" Grid.Column="0" Content="Tip Salariu:"/>

    <StackPanel Grid.Row="2" Grid.Column="1" Orientation="Horizontal">

        <RadioButton x:Name="rbLunarInput" Content="Lunar" GroupName="TipSalariuInput"
Margin="5" Checked="RadioButtonInput_Checked"/>

        <RadioButton x:Name="rbOrarInput" Content="Orar" GroupName="TipSalariuInput"
Margin="5" Checked="RadioButtonInput_Checked"/>

    </StackPanel>

    <!-- Row 3: Salariu Brut / Tarif Oră -->

    <Label Grid.Row="3" Grid.Column="0" Content="Salariu Brut:"/>

    <TextBox Grid.Row="3" Grid.Column="1" x:Name="txtSalariuBrut"
TextChanged="ValidateForm"/>

    <Label Grid.Row="3" Grid.Column="2" Content="Tarif Oră:"/>

    <TextBox Grid.Row="3" Grid.Column="3" x:Name="txtTarifOra"
TextChanged="ValidateForm"/>

    <!-- Row 4: Ore Lucrate and Bonusuri -->

    <Label Grid.Row="4" Grid.Column="0" Content="Ore Lucrate:"/>

    <TextBox Grid.Row="4" Grid.Column="1" x:Name="txtOreLucrate"
TextChanged="ValidateForm"/>

    <Label Grid.Row="4" Grid.Column="2" Content="Bonusuri:"/>

    <TextBox Grid.Row="4" Grid.Column="3" x:Name="txtBonusuri"
TextChanged="ValidateForm"/>

```

```

<!-- Row 5: Data Angajarii and Buttons -->

<Label Grid.Row="5" Grid.Column="0" Content="Data Angajării:"/>

<DatePicker Grid.Row="5" Grid.Column="1" x:Name="dpDataAngajarii" FontSize="14"
Padding="5" SelectedDate="{x:Null}"/>


<StackPanel Grid.Row="5" Grid.Column="2" Grid.ColumnSpan="2"
Orientation="Horizontal" HorizontalAlignment="Right">

    <Button Content="Adaugă" x:Name="btnAdauga" Click="BtnAdauga_Click"

        Background="{StaticResource AccentBrush}" Foreground="White" Padding="15,8"
IsEnabled="False"/>

    <Button Content="Resetează" Click="BtnReset_Click"

        Background="{StaticResource PrimaryBrush}" Foreground="White"
Padding="15,8"/>

</StackPanel>

</Grid>

</GroupBox>

```

Code-Behind with Enhanced Filtering

```

private void AplicaFiltre()
{
    var rezultat = _angajatiOriginali.AsEnumerable();

    // Filter by salary type (RadioButton)
    if (rbLunar.IsChecked == true)
        rezultat = rezultat.Where(a => a.TipSalariu == TipSalariu.Lunar);
    else if (rbOrar.IsChecked == true)
        rezultat = rezultat.Where(a => a.TipSalariu == TipSalariu.Orar);

    // Filter by bonus (CheckBox)
    if (chkBonus.IsChecked == true)

```

```

rezultat = rezultat.Where(a => a.Bonusuri > 0);

// Filter by function (ListBox multiple selection)

var functiiSelectate = lstFunctii.SelectedItems.Cast<ListBoxItem>().Select(item =>
item.Content.ToString()).ToList();

if (functiiSelectate.Any())
{
    rezultat = rezultat.Where(a => functiiSelectate.Contains(a.Functie));
}

// Filter by department (ComboBox)

if (cmbDepartamentFiltru.SelectedItem is ComboBoxItem deptItem &&
deptItem.Content.ToString() != "Toate")
{
    string dept = deptItem.Content.ToString();

    if (Enum.TryParse<Departament>(dept, out Departament departament))
    {
        rezultat = rezultat.Where(a => a.Departament == departament);
    }
}

// Filter by date (DatePicker)

if (dpDataAngajarii.SelectedDate.HasValue)
{
    DateTime dataFiltru = dpDataAngajarii.SelectedDate.Value;

    rezultat = rezultat.Where(a => a.DataAngajarii >= dataFiltru);
}

// Search by name (TextBox)

if (!string.IsNullOrEmpty(txtCautareNume.Text))
{

```

```

string searchTerm = txtCautareNume.Text.Trim();

rezultat = rezultat.Where(a =>

    a.Nume.Contains(searchTerm, StringComparison.OrdinalIgnoreCase) ||

    a.Prenume.Contains(searchTerm, StringComparison.OrdinalIgnoreCase) ||

    a.NumeCompleat.Contains(searchTerm, StringComparison.OrdinalIgnoreCase)

);

}

dgAngajati.ItemsSource = rezultat.ToList();

}

private void LstFunctii_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    AplicaFiltre();
}

private void CmbDepartament_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    AplicaFiltre();
}

private void DpDataAngajarii_SelectedDateChanged(object sender,
SelectionChangedEventArgs e)
{
    AplicaFiltre();
}

private void TxtCautare_TextChanged(object sender, TextChangedEventArgs e)
{
    AplicaFiltre();
}

```


}

Why I Used This Approach

ComboBox for Departments: The ComboBox displays a dropdown list of departments. This eliminates typos and ensures data consistency. The user can only select from predefined values.

DatePicker for Dates: The DatePicker provides a graphical calendar interface for selecting dates. This is much more user-friendly than asking users to type a date in a specific format. It also prevents invalid date entries.

ListBox for Multiple Selection: Setting `SelectionMode="Multiple"` allows users to select multiple job functions by holding Ctrl and clicking. This is useful for filtering by multiple criteria simultaneously.

Dynamic Filter Updates: Each filter control triggers a refresh of the DataGrid, providing immediate feedback to the user.

Combined Filters: All filters work together – the user can combine salary type, bonus, function, department, date, and name search simultaneously.

x:Null for DatePicker: Setting `SelectedDate="{x:Null}"` initializes the DatePicker with no date selected.

Enum Parsing: When the department ComboBox selection changes, the selected string is parsed to the corresponding enum value using `Enum.TryParse`.

Testing & Results

I tested the enhanced filtering:

1. **ComboBox:** Selecting "IT" from the department filter showed only IT employees
2. **DatePicker:** Selecting a date showed only employees hired on or after that date
3. **ListBox:** Selecting multiple functions showed employees who matched any of the selected functions
4. **Combined Filters:** Applying multiple filters at once worked correctly
5. **Input Form:** The ComboBox and DatePicker in the input form worked properly for adding new employees

All controls provided a smooth user experience with immediate feedback.

Reflection

WPF's built-in controls save a huge amount of development time. The ComboBox with enum binding is particularly elegant – I don't need to manually populate items or handle conversion. The DatePicker would have been complex to implement from scratch. The ListBox with multiple selection is a powerful filtering tool that would require significant code in other frameworks. These controls demonstrate WPF's strength in data-driven applications. The ability to combine multiple filters using LINQ's chaining capabilities makes the filtering logic both powerful and maintainable.